Java

27 Nisan 2026 Pazartesi 12:12

```java
package hft12;

// thread'e atanabilir bir sınıf olması için runnable arayüzü implement edilir.
public class KarakterIslem implements Runnable {
    private char karakter;
    private int sayac;

    public KarakterIslem(char karakter, int sayac) {
        this.karakter = karakter;
        this.sayac = sayac;
    }


    // run bir veya birden çok thread tarafından gerçekleştirilebilir
    @Override
    public void run() {
        //karakterleri ekrana bas
        for(int i = 0; i< sayac; i++) {
            System.out.print(karakter);
        }

    }


}
```

```java
package hft12;

public class RakamIslem implements Runnable {
    private int sonSayi;
    public RakamIslem(int sonSayi) {
        this.sonSayi = sonSayi;
    }

    @Override
    public void run() {
        for(int i= 1; i <= sonSayi; i++) {
            System.out.print(i);
        }

    }


}
```

```java
package hft12;

public class Program {

    public static void main(String[] args) {
        // burası seri olmalı çünkü burası işlem tanımlama yeri.
        KarakterIslem aYaz = new KarakterIslem('a', 100);
        KarakterIslem bYaz = new KarakterIslem('b', 100);
        RakamIslem rakamIslem = new RakamIslem(100);

        // her thread oluşturulurken görevleri bellidir ve bu satırlar da seridir.
        Thread thread1 = new Thread(aYaz);
        Thread thread2 = new Thread(bYaz);
        Thread thread3 = new Thread(rakamIslem);

        //paralel mantık burdan sonra artık hangisinin önce biteceği belli değildir.
        thread1.start();
        thread2.start();
        thread3.start();

        while(thread1.isAlive() || thread2.isAlive() || thread3.isAlive()) {
            // ana thread'i oyalıyoruz.
        }

        System.out.println("tüm işlemler kesin olarak bitti.");

    }

}
```

```
aaaabbbbbbbbbb5253545556575859aaaaaaaaaaaaaaaaaaaabbbbbbbbbbb606162aaaaaaaaaaaaaaaaabbb636465666768697071aaaaaaaaaaa727374757677787980818283848586878889909192939495969798991 00tüm işlemler kesin olarak bitti.
```

Thread.Sleep(1) işlemi.

```java
package hft12;

public class RakamIslem implements Runnable {
    private int sonSayi;
    public RakamIslem(int sonSayi) {
        this.sonSayi = sonSayi;
    }

    @Override
    public void run() {
        for(int i= 1; i <= sonSayi; i++) {
            try {
                Thread.sleep(1);
            } catch (InterruptedException e) {
                e.printStackTrace();
            }
            System.out.print(i);
        }

    }


}
```

Ortak havuz yaklaşımı:

```java
public class Program {

    public static void main(String[] args) {
        ExecutorService havuz = Executors.newFixedThreadPool(3); // 3 thread 3 ü de atılan işlerde çalışabilir.
        havuz.execute(new KarakterIslem('a',100));
        havuz.execute(new KarakterIslem('b',100));
        havuz.execute(new RakamIslem(100));

 *      //eski örnek

        havuz.shutdown(); // ana threadte bu yazılsa dahi aslında işlemler bitene kadar devam eder sonra kapanır havuz.

        while(!havuz.isTerminated()) {
            //işlemler bitmediyse ana thread'i oyala
        }

        System.out.println("tüm işlemler kesin olarak bitti.");

    }

}
```

Parallelleştirme problemi. senkranizasyon sıkıntısı ve yazma kümesinin çakışması.

```java
package hft12;

//hesapta veriler tutulduğu için asıl işlem yapılan sınıf değildir runnable imp mantıksız olur.
//ParaYatirma diye ayrı bir sınıf yapıcaz
public class Hesap {
    private int bakiye;
    public Hesap() {
        bakiye = 0;
    }

    //kesinlikle paralel yapılmamalıdır çünkü 1 thread parayı okurken aynı parayı başka bir thread'te aynı sayıyı okur.
    public void paraYatir(int miktar) {
        int yeniToplam = bakiye + miktar;
        try {
            Thread.sleep(1);
        } catch (InterruptedException e) {
            // TODO Auto-generated catch block
            e.printStackTrace();
        }
        bakiye = yeniToplam;
    }

    public int getBakiye() {
        return bakiye;
    }
```

```java
package hft12;

public class ParaYatirma implements Runnable {
    private Hesap hesap;
    private int miktar;

    public ParaYatirma(Hesap hesap, int miktar) {
        this.hesap = hesap;
        this.miktar = miktar;
    }

    @Override
    public void run() {
        // TODO Auto-generated method stub
        hesap.paraYatir(miktar);

    }
}
```

```
Toplanan Yardım Parası: 21
```

100 olmalı fakat 21 olmuş

```java
package hft12;

import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;

public class ProgramHesapOrn {

    public static void main(String[] args) {
        Hesap yardimKasasi = new Hesap();
        ExecutorService havuz = Executors.newFixedThreadPool(5);
        for(int i = 0; i< 100; i++) {
            havuz.execute(new ParaYatirma(yardimKasasi, 1));
        }
        havuz.shutdown();
        while(!havuz.isTerminated()) {

        }
        System.out.println("Toplanan Yardım Parası: " + yardimKasasi.getBakiye());
    }

}
```

Kritik Bölge Çözümü: lock ve unlock:

```java
import java.util.concurrent.locks.Lock;
import java.util.concurrent.locks.ReentrantLock;
```

```java
public class Hesap {
    private int bakiye;
    private final Lock bolge;
    public Hesap() {
        bolge = new ReentrantLock();
        bakiye = 0;
    }

    // bolge.lock ve unlock() ile burayı kritik bölge belirleyip
    public void paraYatir(int miktar) {
        bolge.lock();
        int yeniToplam = bakiye + miktar;
        try {
            Thread.sleep(1);
        } catch (InterruptedException e) {
            // TODO Auto-generated catch block
            e.printStackTrace();
        }
        bakiye = yeniToplam;
        bolge.unlock();
    }

    public int getBakiye() {
        return bakiye;
    }
}
```